Case Study: ระบบร้านกาแฟ Multi-Threaded

สถานการณ์ : ร้านกาแฟมีลูกค้าหลายคนสั่งเครื่องดื่มพร้อมกัน บาริสต้าหลายคนสามารถชงกาแฟพร้อมกันได้ โดยใช้ Thread จำลองว่า:

- 1 Order = 1 Thread
- การชงใช้เวลา 2 วินาที
- หลายเมนูทำพร้อมกันได้

Phase 1: หลายออเดอร์ทำพร้อมกัน

```
public class CoffeeShop {
    public static void main(String[] args) {

        Thread t1 = new Thread(new CoffeeMaker("Espresso"));
        Thread t2 = new Thread(new CoffeeMaker("Latte"));
        Thread t3 = new Thread(new CoffeeMaker("Cappuccino"));

        t1.start();
        t2.start();
        t3.start();

        System.out.println("☕ รับออเดอร์ครบแล้ว");
    }
}
```

คำถามอภิปราย (อธิบาย)

1. ทำไมลำดับการเสร็จไม่เหมือนเดิมทุกครั้ง?
   ตอบ เพราะ ใช้ start() ทุก Thread จึงทำงานพร้อมกัน และ JVM ไม่รับประกันลำดับการทำงาน

2. ถ้าเรียก run() แทน start() จะเกิดอะไรขึ้น?
   ตอบ จะไม่เกิด Thread ใหม่ จะทำงานใน main Thread ตัวเดียว และจะทำทีละออเดอร์, ไม่ทำพร้อมกันและลำดับเหมือนเดิมทุกครั้ง

3. main thread ทำงานพร้อมกับ CoffeeMaker หรือไม่?
   ตอบ ทำงานพร้อมกัน เพราะ Java จะสร้าง Thread ใหม่ 3 ตัว และ main thread ยังทำงานต่อได้ทันที

Phase 2: ปัญหาเครื่องชงมีเครื่องเดียว (Race Condition)

ร้านมี “เครื่องชงกาแฟแค่ 1 เครื่อง” แต่หลาย thread พยายามใช้พร้อมกัน

```
class CoffeeMachine {
    public void brew(String menu) {
        System.out.println(" กำลังใช้เครื่องชง: " + menu);
        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {}
        System.out.println("เครื่องดื่มเสร็จ: " + menu);
    }
}
```

ปรับ CoffeeMaker ให้ใช้เครื่องเดียวกัน

```
class CoffeeMaker implements Runnable {
    private String menu;
    private CoffeeMachine machine;

    public CoffeeMaker(String menu, CoffeeMachine machine) {
        this.menu = menu;
        this.machine = machine;
    }

    @Override
    public void run() {
        machine.brew(menu);
    }
}
```

และในส่วน main

```
public class CoffeeShop {
    public static void main(String[] args) {

        CoffeeMachine machine = new CoffeeMachine();

        new Thread(new CoffeeMaker("Espresso", machine)).start();
        new Thread(new CoffeeMaker("Latte", machine)).start();
        new Thread(new CoffeeMaker("Mocha", machine)).start();
    }
}
```

ปัญหา

เครื่องเดียวถูกใช้พร้อมกัน (แม้จริง ๆ ต้องใช้ทีละคน)

แบบฝึกหัดจากกรณีศึกษา

1. เพิ่มจำนวนเมนูเป็น 10 รายการ แล้วสังเกตชื่อ thread java

```
public class CoffeeShop {
    public static void main(String[] args) {

        CoffeeMachine machine = new CoffeeMachine();

        new Thread(new CoffeeMaker("Espresso", machine)).start();
        new Thread(new CoffeeMaker("Latte", machine)).start();
        new Thread(new CoffeeMaker("Mocha", machine)).start();
        new Thread(new CoffeeMaker("Americano", machine)).start();
        new Thread(new CoffeeMaker("Birdy", machine)).start();
        new Thread(new CoffeeMaker("Macchiato", machine)).start();
        new Thread(new CoffeeMaker("Arabica", machine)).start();
        new Thread(new CoffeeMaker("Flat White", machine)).start();
        new Thread(new CoffeeMaker("Affogato", machine)).start();
        new Thread(new CoffeeMaker("Dirty Coffee", machine)).start();
    }
}
```

ตอบ สังเกตได้ว่าชื่อ Thread ไม่เรียงตามลำดับ

2. เปลี่ยน Thread.sleep(2000) เป็นสุ่ม Math.random()*3000 แล้วดูความสลับของ output
java
```
class CoffeeMachine {

    public void brew(String menu) {

        System.out.println("กำลังใช้เครื่องชง: " + menu);

        try {
            Thread.sleep((int)(Math.random() * 3000));
        } catch (InterruptedException e) {}

        System.out.println("เครื่องดื่มเสร็จ: " + menu);
    }
}
```

ตอบ สังเกตได้ว่า เวลาแต่ละ Thread ไม่เท่ากันและ ลำดับเสร็จจะสลับทุกครั้ง

3. ทดลองเรียก run() แทน start() แล้วอธิบายผล java
```
public class CoffeeShop {
    public static void main(String[] args) {

        CoffeeMachine machine = new CoffeeMachine();

        new Thread(new CoffeeMaker("Espresso", machine)).run();
        new Thread(new CoffeeMaker("Latte", machine)).run();
        new Thread(new CoffeeMaker("Mocha", machine)).run();
        new Thread(new CoffeeMaker("Americano", machine)).run();
        new Thread(new CoffeeMaker("Birdy", machine)).run();
        new Thread(new CoffeeMaker("Macchiato", machine)).run();
        new Thread(new CoffeeMaker("Arabica", machine)).run();
        new Thread(new CoffeeMaker("Flat White", machine)).run();
        new Thread(new CoffeeMaker("Affogato", machine)).run();
        new Thread(new CoffeeMaker("Dirty Coffee", machine)).run();
    }
}
```

ตอบ สังเกตได้ว่า ไม่สร้าง Thread ใหม่ และทำใน main thread ทำทีละออเดอร์ และ output จะเรียงเสมอ

4. ใส่ System.out.println("End main") ต่อท้าย แล้วดูว่า main จบก่อนชงเสร็จหรือไม่ java

```
public class CoffeeShop {
    public static void main(String[] args) {

        CoffeeMachine machine = new CoffeeMachine();

        new Thread(new CoffeeMaker("Espresso",
machine)).start();
        new Thread(new CoffeeMaker("Latte",
machine)).start();
        new Thread(new CoffeeMaker("Mocha",
machine)).start();
        new Thread(new CoffeeMaker("Americano",
machine)).start();
        new Thread(new CoffeeMaker("Birdy",
machine)).start();
        new Thread(new CoffeeMaker("Macchiato",
machine)).start();
        new Thread(new CoffeeMaker("Arabica",
machine)).start();
        new Thread(new CoffeeMaker("Flat White",
machine)).start();
        new Thread(new CoffeeMaker("Affogato",
machine)).start();
        new Thread(new CoffeeMaker("Dirty Coffee",
machine)).start();

        System.out.println("End main");
    }
}
```

ตอบ สังเกตได้ว่า เมื่อใส่ท้าย main main จะพิมพ์ก่อน แล้ว thread ยังทำต่อ

5. แก้ปัญหาโดยใช้ synchronized และ AtomicInteger java

```
import java.util.concurrent.atomic.AtomicInteger;

class CoffeeMachine {

    private AtomicInteger counter = new AtomicInteger(0);

    public synchronized void brew(String menu) {

        int id = counter.incrementAndGet();

        System.out.println(
            "Order " + id + " กำลังใช้เครื่อง: " + menu
        );

        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {}

        System.out.println(
            "Order " + id + " เสร็จ: " + menu
        );
    }
}
```

ตอบ synchronized ป้องกันการใช้เครื่องพร้อมกัน และ AtomicInteger ใช้นับค่าแบบปลอดภัยใน multithread